VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 8: Sorting Algorithm

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong
Dr. LE Huu Ton

What is sorting algorithm?

Sorting

- ***Sorting*** is a process that organizes a collection of data into either ascending or descending order.
- Formally
- Input: A sequence of n numbers <a1,a2,…,an>
- Output: A reordering <a’1,a’2,…,a’n> of the sequence such that a’1 ≤ a’2 ≤ … ≤ a’n
- Given the input <6, 3, 1, 7>, the algorithm should produce
<1, 3, 6, 7>
- Called an instance of the problem

Sorting

We encounter sorting almost everywhere:

- Sorting prices from lowest to highest
- Sorting flights from earliest to latest
- Sorting grades from highest to lowest
- Sorting songs based on artist, album, playlist, etc.

Sorting Algorithm

- There are many sorting algorithms (as of 27.10.2014 there are 44 Wikipedia entries)
- In this class we will learn:
  - Selection Sort
  - Insertion Sort
  - Bubble Sort

Selection Sort

- Partition the input list into a sorted and unsorted part (initially sorted part is empty)
- **Select** the smallest element and put it to the end of the sorted part
- Increase the size of the sorted part by one
- Repeat this n-1 times to sort a list of n elements

Selection Sort

Sorted			Unsorted			
23	78	45	8	32	56	Original List
8	78	45	23	32	56	After pass 1
8	23	45	78	32	56	After pass 2
8	23	32	78	45	56	After pass 3
8	23	32	45	78	56	After pass 4
8	23	32	45	56	78	After pass 5

Selection Sort

```
void selection_sort(float array[], long n){
    int i, j, position;
    float temp;
    for (i = 0; i < (n - 1); i++){          // finding minimum element (n-1) times
        position = i;                       // set initial smallest position
        for (j = i + 1; j < n; j++){
            if (array[position] > array[j])
                position = j;                 // found the smallest position
        }
        if (position != i){
            temp = array[i];
            array[i] = array[position];
            array[position] = temp;
        }
    }
}
```

Selection Sort

- What is the complexity of selection sort?
- Does it have different best, average, and worst case complexities?

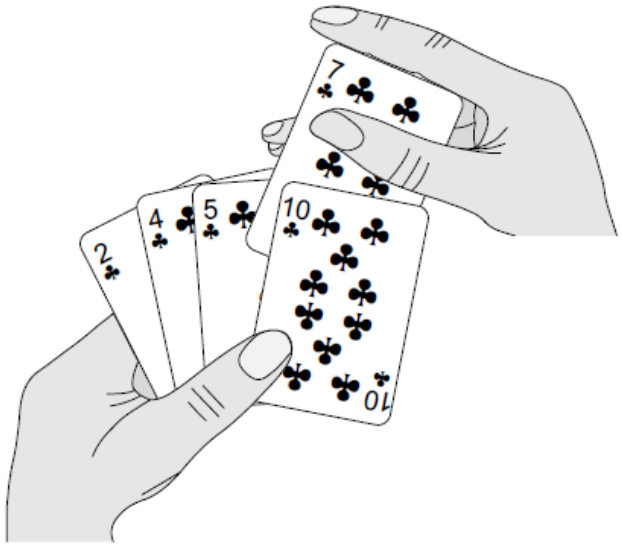
Selection Sort

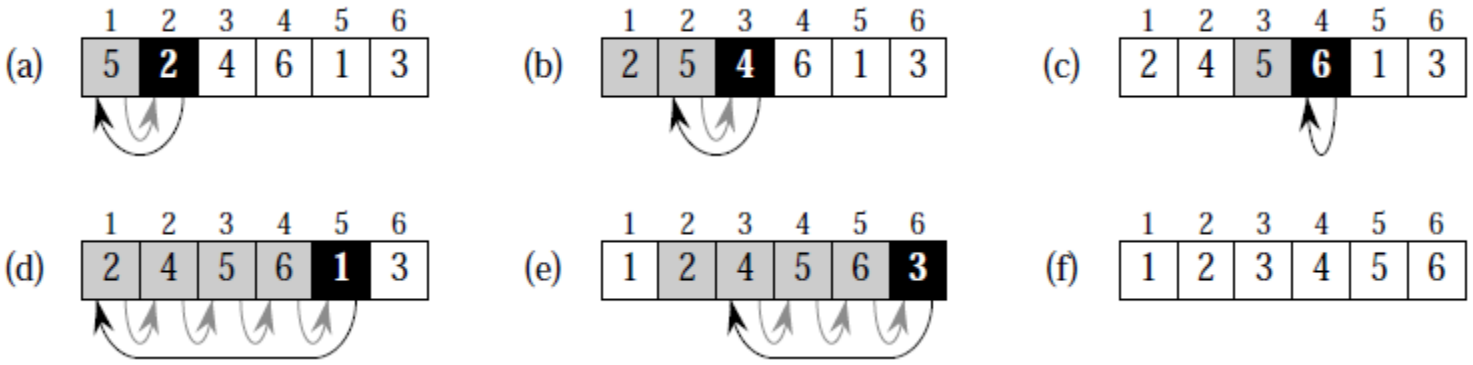
- Selection sort is $O(n^2)$ for all three cases (prove this)
- Therefore it is not very efficient

Insertion sort

- Insertion sort is a simple sorting algorithm that is appropriate for small inputs.
  - Most common sorting technique used by card players.
- Again, the list is divided into two parts: sorted and unsorted.
- In each pass, the first element of the unsorted part is picked up, transferred to the sorted sublist, and **inserted** at the appropriate place.
- A list of *n* elements will take at most *n-1* passes to sort the data.

Insertion sort





Insertion sort

Sorted			Unsorted			
23	78	45	8	32	56	Original List
23	78	45	8	32	56	After pass 1
23	45	78	8	32	56	After pass 2
8	23	45	78	32	56	After pass 3
8	23	32	45	78	56	After pass 4
8	23	32	45	56	78	After pass 5

Insertion sort

```
void insertion_sort(float array[], long n){
    int i,j, flag=0;
    float temp;

    for (i = 1 ; i <= n - 1; i++) {
        temp = array[i];
        for (j = i - 1 ; j >= 0; j--) {
            if (array[j] > temp) {
                array[j+1] = array[j];
                flag = 1;
            }
            else
                break;
        }
            if (flag){
                array[j+1] = temp;
            }
    }
}
```

Insertion sort

- Running time depends on not only the size of the array but also the contents of the array.
- ***Best-case:*** □ **O(n)**
  - Array is already sorted in ascending order.
- ***Worst-case:*** □ **$O(n^2)$**
  - Array is in reverse order:
- ***Average-case:*** □ **$O(n^2)$**
  - We have to look at all possible initial data organizations.

Insertion sort

- Which running time will be used to characterize this algorithm?
  - Best, worst or average?
- Worst:
  - Longest running time (this is the upper limit for the algorithm)
  - It is guaranteed that the algorithm will not be worse than this.
- Sometimes we are interested in average case. But there are some problems with the average case.
  - It is difficult to figure out the average case. i.e. what is average input?
  - Are we going to assume all possible inputs are equally likely?
  - In fact for most algorithms average case is same as the worst case.

Bubble sort

- Repeatedly swap adjacent elements that are out of order.

- Traverse from left and compare adjacent elements and the higher one is placed at right side.

- In this way, the largest element is moved to the rightmost end at first.

- This process is then continued to find the second largest and place it and so on until the data is sorted.

Bubble sort

23	78	45	8	32	56
23	45	78	8	32	56
23	45	8	78	32	56
23	45	8	32	78	56
23	45	8	32	56	78
23	8	45	32	56	78
8	23	45	32	56	78
8	23	32	45	56	78

Bubble sort

```
void bubble_sort(float array[], long n){
    long i, j;
    float swap;

    for (i = 0 ; i < n - 1; i++){
        for (j = 0 ; j < n - i - 1; j++){
            if (array[j] > array[j+1]){       /* For decreasing order use '<'
instead of '>' */
                /* Swapping */
                swap      = array[j];        // save temporary in swap
                array[j]   = array[j+1];    //  go to the next element
                array[j+1] = swap;          //  set swap in the j+1.
            }
        }
    }
}
```

Bubble sort

- ***Best-case:*** □ **O(n)**
  - Array is already sorted in ascending order.
- ***Worst-case:*** □ **$O(n^2)$**
  - Array is in reverse order:
- ***Average-case:*** □ **$O(n^2)$**
  - We have to look at all possible initial data organizations.

- In fact, any sorting algorithm which sorts elements by swapping **adjacent** elements can be proved to have an $O(n^2)$ average case complexity

Practice

Modify the 3 sorting algorithms to sort an array in descending order

Sorting and searching

Write a program to find a number in an array, if the number is in the array, return the index of that number in the array.

Consider two cases: the array is not sorted and the array is already sorted. How can you speed up the searching process if the array is already sorted?